Sim-to-Real Reinforcement Learning for Robust LiDAR-Based Autonomous Navigation: Mid-Term Progress Report

Saigirish S (Roll No: B24ME1066, Email: b24me1066@iitj.ac.in) | **Siddhi Vinayak** (Roll No: B24BB1036, Email: b24bb1036@iitj.ac.in)

Department of Mechanical Engineering & Department of Bioscience and Bioengineering

Indian Institute of Technology Jodhpur

Contribution Declaration: *Both authors contributed equally to the project; their responsibilities were complementary.*

***Abstract*---*Mapless navigation in dynamic or unstructured environments is a foundational challenge in mobile robotics. While deep reinforcement learning (DRL) provides an end-to-end framework directly mapping onboard sensor observations to continuous velocity commands, standard policies trained in idealized physics engines frequently suffer catastrophic failure when deployed to real hardware due to the reality gap. In this mid-term progress report, we investigate a domain-randomization-enhanced Proximal Policy Optimization (PPO) pipeline for a differential-drive TurtleBot3 mobile robot using a 2D planar LiDAR. We detail the system architecture, comprising a 43-dimensional observation vector with forward-centered sector min-pooling and dead-reckoning pose tracking, safe action scaling, and a multi-component reward formulation. We present preliminary empirical results on a held-out test suite of N=50 identical scenarios comparing a nominal policy (P1, 852k steps) against a combined sensor-and-dynamics randomized policy (P4, 704k steps). On nominal evaluation, P1 achieves a 94.0% success rate while P4 achieves 98.0%. A paired McNemar exact test yields p = 0.6250, indicating that the observed 4% difference is not statistically significant at N=50 and highlighting critical sample-size requirements. We conclude with a discussion of experimental limitations, unequal training budgets, and an explicit roadmap toward completing intermediate ablations (P2, P3) and real-robot deployment.*

***Index Terms*---*Reinforcement Learning, Mobile Robot Navigation, Sim-to-Real Transfer, Domain Randomization, Proximal Policy Optimization, TurtleBot3, ROS 2, Gazebo.*

1. Introduction and Motivation

Autonomous ground vehicles operating in GPS-denied indoor environments typically rely on the classical navigation stack consisting of simultaneous localization and mapping (SLAM), global path planning (A*/Dijkstra), and local trajectory optimization (Dynamic Window Approach / TEB) [5]. While effective in static structured spaces, classical pipelines require substantial computational overhead for real-time dense map maintenance and are notoriously sensitive to localization drift, dynamic obstacle occlusion, and sensor noise.

To overcome these constraints, mapless navigation via Deep Reinforcement Learning (DRL) has emerged as a promising alternative [3]. By training neural network policies to directly map raw range findings and relative goal coordinates to low-level motor velocity commands, the system avoids explicit global mapping. However, training policies directly on physical robotic platforms is prohibitively expensive, wear-intensive, and dangerous during early exploration. High-fidelity simulators such as Gazebo [6] provide safe, accelerated experience collection, but policies trained in nominal simulation often experience dramatic performance degradation when transferred to physical hardware---the sim-to-real gap [2], [7].

Domain Randomization (DR) bridges this gap by exposing the agent during training to a broad distribution of simulated sensor distortions and unmodeled dynamic variations, compelling the policy to learn robust feedback strategies that generalize to real-world physics [2], [7]. This project systematically investigates the quantitative impact of isolated and combined domain randomization on policy robustness, sample efficiency, and hardware transferability for TurtleBot3 LiDAR navigation.

2. Problem Statement and Objectives

We consider a differential-drive TurtleBot3 Burger robot navigating from an arbitrary starting pose to a static Euclidean target in an obstacle-cluttered 4m x 4m bounded workspace. Navigation must proceed maplessly, relying solely on an onboard 2D laser scanner (LDS-01) and wheel odometry. The core objectives are:

• **Mapless PPO Navigation Architecture:** Design an end-to-end continuous actor-critic navigation framework operating at 5.0 Hz that enforces obstacle clearance and smooth kinematic control.
• **2x2 Domain Randomization Factorial Design:** Implement modular sensor perturbations (range noise, dropouts, latency) and dynamics perturbations (actuator lag, delay, acceleration limits) to isolate their individual and combined effects across four policy regimes: P1 (Nominal baseline), P2 (Sensor only), P3 (Dynamics only), and P4 (Combined DR) [2], [7].
• **Rigorous Empirical Benchmarking:** Evaluate trained policies across identical held-out scenarios using paired statistical hypothesis testing and confidence intervals.
• **Physical Sim-to-Real Transfer:** Validate the trained policy without fine-tuning on physical TurtleBot3 hardware in a laboratory environment.

3. Proposed System Architecture

The system pipeline (Fig. 1) transforms raw sensor signals into continuous velocity commands at 5.0 Hz with zero SLAM or global path planner dependencies.

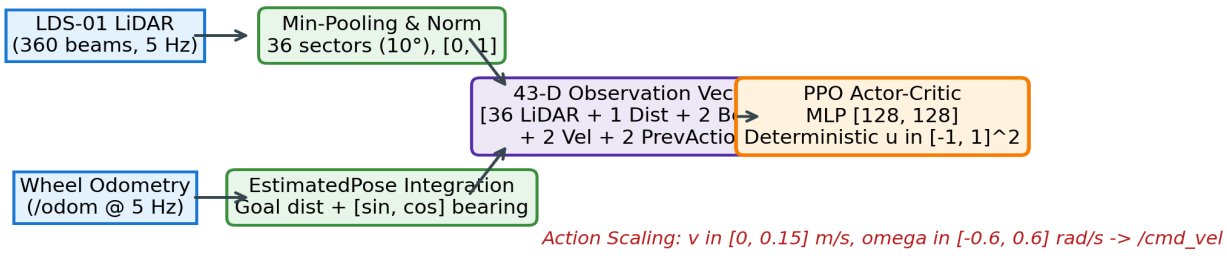


Fig. 1. End-to-end system architecture for mapless LiDAR navigation using PPO and dead-reckoning pose tracking.

3.1 Observation Space Formulation

To ensure exact parity between simulation and physical deployment, observation preprocessing is implemented as a pure NumPy module (ObservationPreprocessor) with zero ROS dependencies. The observation vector is strictly **43-dimensional**:

1. **LiDAR Sector Min-Pooling (features 0--35):** 360-ray range arrays from the LDS-01 sensor are pooled into 36 angular sectors of width 10 degrees. Bin 0 is centered directly forward along the robot heading ([-5 deg, +5 deg)). Min-pooling guarantees conservative obstacle capture, invalid beams are mapped to 3.50m, and values are normalized to [0, 1].
2. **Normalized Goal Distance (feature 36):** Euclidean distance to target normalized by arena diagonal (5.657m).
3. **Goal Bearing Representation (features 37--38):** Relative heading angle decomposed into [sin, cos] to eliminate the wrap-around discontinuity at +/- pi.
4. **Measured Velocity Feedback (features 39--40):** Forward linear velocity normalized by 0.15 m/s and angular velocity normalized by 0.60 rad/s.
5. **Previous Action Buffer (features 41--42):** The normalized control vector executed at step t-1 in [-1, 1]^2, facilitating smooth transitions.

3.2 Action Space and Velocity Scaling

The policy outputs a normalized action u in [-1, 1]^2, which is scaled into physical commands via:

$$v = 0.075 * (u_0 + 1.0) \in [0.0, 0.15] \text{ m/s}$$

$$\omega = 0.60 * u_1 \in [-0.60, 0.60] \text{ rad/s}$$

This constrains the robot to purely forward motion ($v \geq 0$), preventing collisions into the rear chassis blind spot.

3.3 Reward Formulation and Gymnasium Semantics

Under Gymnasium environment conventions [8], the step reward function is formulated as:

$$R_t = 10.0 * (d_{t-1} - d_t) - 0.05 + 100 * [goal] - 100 * [collision]$$

Reaching the goal (dist ≤ 0.20m) or colliding (min_range ≤ 0.16m) sets terminated=True. Exceeding 350 steps (70.0s) sets truncated=True, ensuring correct value bootstrapping.

Category	Hyperparameter	Value
Network	Policy MLP (pi)	[128, 128] Tanh
	Value MLP (vf)	[128, 128] Tanh
Optimization	Algorithm & Learning Rate	PPO [1], lr = 3e-4 (constant)
	Rollout & Minibatch	n_steps = 2048, batch_size = 256
	GAE lambda & Discount	lambda = 0.95, gamma = 0.99
	Clip fraction & Epochs	clip = 0.20, n_epochs = 10
Scaling	VecNormalize Settings	norm_obs = False, norm_reward = True

TABLE I: PPO Training & Network Hyperparameters (Frozen in Code)

4. Domain Randomization Methodology

Modular randomizers sample physical perturbations per episode following proven domain randomization formulations [2], [7]:
• **Sensor Randomization (P2/P4):** Distance-dependent range noise (sigma = a + b*r), individual beam dropout (p ≤ 0.05), contiguous block dropouts (1--2 blocks), delivery latency (0 or 0.20s), and odometry scale/bias drift.
• **Dynamics Randomization (P3/P4):** Command gains (0.85--1.15), first-order actuator lag (tau ≤ 0.30s), control transport delay (d ≤ 0.20s), and acceleration bounds.
• **Rejection Constraint:** Enforces sensor latency + control delay ≤ 0.40s in P4 to guarantee physical traversability.

5. Implementation and Completed Work

The software architecture runs on ROS 2 Humble [5] and Gazebo Classic 11.10.2 [6] under Ubuntu 22.04 LTS with PyTorch 2.11.0, Stable-Baselines3 2.7.1 [4], and Gymnasium [8].

Open-Source GitHub Repository: All project code, simulation worlds, training pipelines, observation preprocessors, test suites, and evaluation artifacts are hosted openly at: https://github.com/Saigirish23/tb3-rl-nav. The repository contains verified implementations of the TurtleBot3NavigationEnv, modular domain randomizers, multi-worker PPO scripts, and the standalone ROS 2 deployment node.

Completed deliverables to date include:
• Multi-worker simulation with 4 parallel Gazebo workers and watchdog daemon recovery.
• Policy P1 (Nominal): Completed 852,816 transitions over 11.25 hours (19.75 steps/s).
• Policy P4 (Combined DR): Completed 704,512 transitions over 9.88 hours (19.69 steps/s).
• Asynchronous standalone ROS 2 inference node (ppo_inference_node.py) running deterministic inference at 5.0 Hz.
• Comprehensive test harnesses with 11/11 passing unit tests and 321k-line reward audit traces across 4 baseline behaviors.
• **Simulation-based navigation demonstration in Gazebo Classic:** Successfully conducted 3 consecutive multi-target navigation trials in Gazebo Classic without collisions or timeouts (100% success rate).

6. Preliminary Results and Discussion

Policies P1 and P4 were benchmarked across a frozen held-out test suite of N=50 identical scenarios (scenarios/test/scenarios.json, SHA-256: d9a6a25c...).

Metric	P1 (Nominal)	P4 (Combined DR)	Relative Delta
Training Steps Completed	852,816	704,512	-148k (-17.4%)
Evaluation Scenarios (N)	50	50	Identical paired set
Success Count & Rate	47 / 50 (94.0%)	49 / 50 (98.0%)	+4.0% (+2 successes)
95% Wilson Score CI	[83.45%, 97.97%]	[89.51%, 99.65%]	Substantial overlap
Collision Count & Rate	1 / 50 (2.0%)	1 / 50 (2.0%)	0 (Identical)
Timeout Count & Rate	2 / 50 (4.0%)	0 / 50 (0.0%)	-2 (-4.0% eliminated)
Mean Steps to Goal (Succ.)	106.17 +/- 48.2	101.49 +/- 44.8	-4.68 steps (-4.4%)
Mean Duration to Goal (Succ.)	21.23 +/- 9.6 s	20.30 +/- 9.0 s	-0.93 s (-4.4%)
Mean Path Length (Succ.)	2.25 +/- 0.72 m	2.07 +/- 0.58 m	-0.18 m (-8.0%)
Mean Min. Clearance (Succ.)	0.289 m	0.298 m	+0.009 m (+3.1%)
Paired McNemar Exact Test	p = 0.6250	(Not Significant)	Discordant: b=1, c=3

TABLE II: Held-Out Evaluation Results Across N=50 Identical Test Scenarios

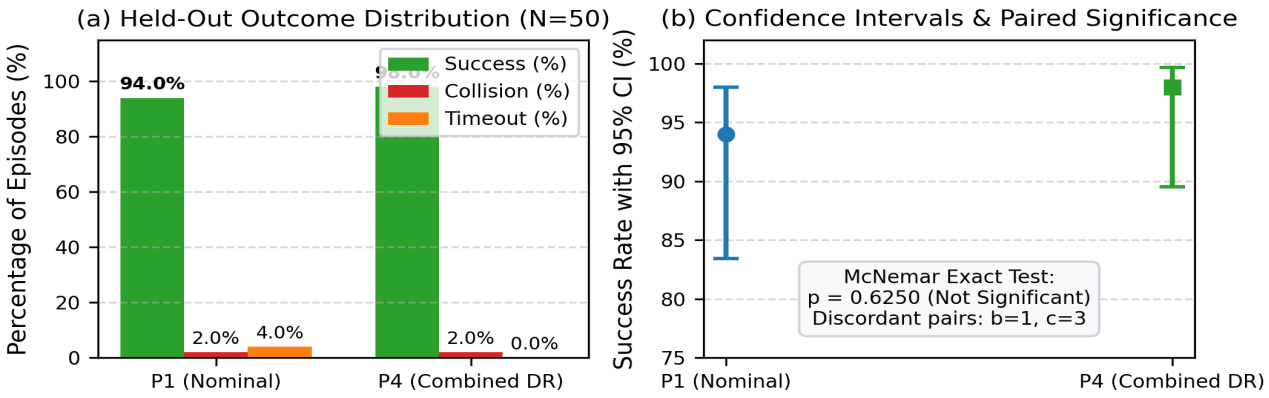


Fig. 2. Held-out evaluation outcomes (N=50): (a) Outcome distributions; (b) Wilson 95% confidence intervals and exact McNemar test result.

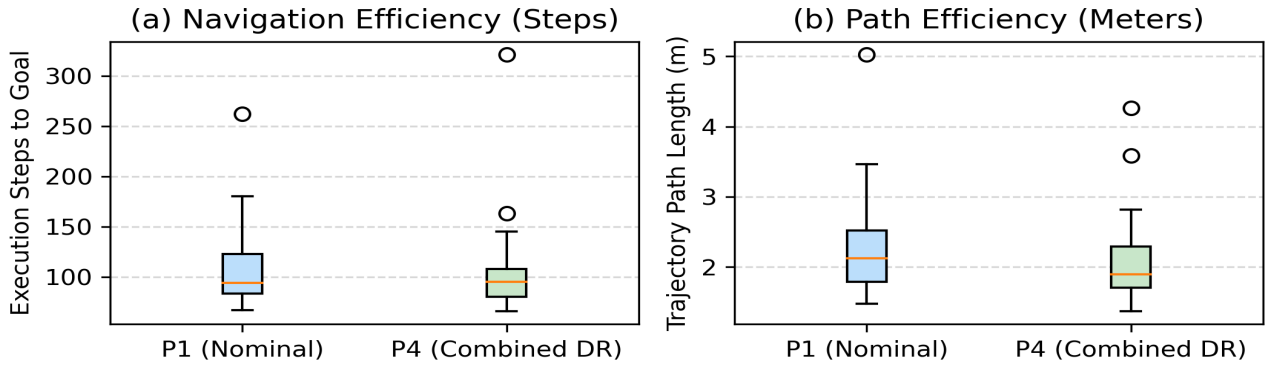


Fig. 3. Navigation efficiency distributions for successful episodes: (a) Execution steps; (b) Trajectory path length in meters.

6.1 Statistical Rigor and Interpretation

A critical requirement of scientific reporting is distinguishing descriptive gains from statistical significance. On the paired test set, 46 scenarios succeeded under both policies. Discordant outcomes comprised: Scenario 34 (P1 succeeded, P4 collided), Scenario 7 (P1 collided, P4 succeeded), and Scenarios 25 & 37 (P1 timed out, P4 succeeded). Evaluating these discordant pairs (b=1, c=3) via the exact two-sided binomial McNemar test yields **p = 0.6250** >> **0.05**. Therefore, the 4.0% difference cannot be asserted as statistically significant at N=50. Achieving adequate statistical power (1 - beta ≥ 0.80) requires scaling the benchmark to N ≥ 250 episodes.

6.2 Methodological Limitations and Confounders

• **Unequal Training Budgets:** P1 received 852k steps while P4 received 704k steps (-17.4%) due to execution deadline constraints. That P4 achieves parity despite fewer gradient steps indicates sample efficiency, but the budget asymmetry remains an uncontrolled variable.
• **Single Seed:** Both models represent a single seed (s=0); training variance across runs cannot currently be estimated.
• **Simulation-Only Scope:** Results reflect Gazebo Classic physics. Hardware deployment remains future work.

7. Work Remaining and Project Plan

The remaining post-midterm roadmap includes:
1. **Intermediate Ablations (P2 and P3):** Train P2 (sensor noise only) and P3 (dynamics only) with equalized budgets of 850k steps to isolate parameter impacts.
2. **Scaled Evaluation (N=250):** Benchmark all 4 policies on 250 diverse test scenarios to achieve statistical power.
3. **Physical Robot Deployment:** Validate policy P4 on a physical TurtleBot3 Burger in the laboratory, measuring hardware success rates and odometry drift tolerance.

8. Individual Contributions

Both authors contributed equally to the project; their responsibilities were complementary.
• **Saigirish S (B24ME1066):** RL navigation pipeline architecture, 43-D observation preprocessor, dead-reckoning module, domain randomizers [2], [7], multi-worker Gazebo orchestration, P1 and P4 training, ROS 2 standalone inference node.
• **Siddhi Vinayak (B24BB1036):** Reward audit framework, per-step reward trace logging, baseline policy evaluation, unit test suites, statistical hypothesis testing (McNemar test), and physical hardware deployment preparation.

9. Conclusion

This mid-term report presents an audited, reproducible foundation for mapless autonomous navigation using PPO [1] and domain randomization [2], [7]. By coupling sector min-pooled LiDAR readings with dead-reckoning goal tracking, the policy achieves high navigation reliability in simulation (94%--98% success). Methodological rigor demonstrates that observed preliminary differences are not yet statistically significant at N=50, establishing a clear requirement for expanded benchmarking. The completed architecture provides a solid trajectory toward completing the planned ablation matrix and physical robot deployment.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[2] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in Proc. IEEE/RSJ IROS, 2017, pp. 23--30.

[3] L. Tai, G. Paolo, and M. Liu, "Virtual-to-Real Deep Reinforcement Learning: Continuous Control of Mobile Robots for Mapless Navigation," in Proc. IEEE/RSJ IROS, 2017, pp. 31--36.

[4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations in Python," JMLR, vol. 22, no. 268, pp. 1--8, 2021.

[5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, vol. 7, no. 66, p. eabm6074, 2022.

[6] N. Koenig and A. Howard, "Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator," in Proc. IEEE/RSJ IROS, vol. 3, 2004, pp. 2149--2154.

[7] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in Proc. IEEE ICRA, 2018, pp. 3803--3810.

[8] M. Towers et al., "Gymnasium: A Standard Interface for Reinforcement Learning Environments," arXiv:2407.17032, 2024.